Answer 1:

1. System Log:

[start_transaction, T_1]
[read_item, T_1, D]
[write_item, T_1, D, 20,25]
[commit, T_1]
[start_transaction, T_2]
[read_item, T_2, B]
[write_item, T_2, B, 120, 180]
[start_transaction, T_4]
[read_item, T_4, D]
[write_item, T_4, D, 25, 115]
[start_transaction, T_3]
[read_item, T_3, C]
[write_item, T_3, C, 30, 40]
[read_item, T_4, A]
[write_item, T_4, A, 30, 20]
[commit, T_3]
[read_item, T_4, A]
[write_item, T_4, A, 20, 15]

1. A =30 B=120, C=40, D = 25
2. T1 and T3 are redone and T2 and T4 are rolled back.

Answer 2:

1. Wait-Die: Wait- Die is a non preemptive scheme. As the name specifies, the transaction either waits for the lock or dies. The decision on whether to wait or die is made based on the time stamp. A transaction T2 time stamp is greater than the time stamp value of a transaction T1 currently using the lock, then T2 cannot wait and is rolled back.
2. Wound – Wait: It is a preemptive scheme wherein if a transaction Ti requests a dataitem which is currently under the control of transaction Tj, it is allowed to wait if its time stamp value is greater than that of Tj, else Tj is rolled back.

Answer 3:

i. Which schedule(s) is/are strict, serial schedule?
S3

ii. Is Schedule S3 recoverable?
Yes

iii. Is schedule S4 cascadeless?
Yes

iv. Which pair of schedules are conflict equivalent? Just give one such schedule.
S1 and S2

v. Is schedule S5 a recoverable schedule?
Yes

vi. Are schedule S1 and S5 result equivalent?

Yes

vii. Is schedule S6 cascadeless?

No

viii. Which pair of schedules are view equivalent? Just give one such schedule.

S5 and S6

Answer 4:

Solution:

First Normal Form:

Course_ ID	Student_ ID	Student_ Name	Student_Age	Course_Name	Classroom	Faculty_ ID	Faculty_ Name	Faculty_Contact
CS2005	21k-4779	Taha	21	Database Systems	R12	5773	Ms. Tania	0300-2233446
CS2005	21k-4504	Rafay	22	Database Systems	R12	5773	Ms. Tania	0300-2233446
CS2005	21k-4518	Sarim	22	Database Systems	R12	5773	Ms. Tania	0300-2233446
CS3001	21k-4701	Amna	21	Computer Networks	E2	9100	Mr. Ali	0333-2130523
CS3001	21k-4732	Bilal	22	Computer Networks	E2	9100	Mr. Ali	0333-2130523
CS3001	21k-4702	Shahid	21	Computer Networks	E2	9100	Mr. Ali	0333-2130523
CS2001	21k-4167	Abu Bakr	23	Algorithms	E5	5218	Mr. Rahul	0345-4897524
CS2001	21k-4478	Saad	20	Algorithms	E5	5218	Mr. Rahul	0345-4897524
CS2001	21k-4856	Moiz	21	Algorithms	E5	5218	Mr. Rahul	0345-4897524
CY3110	21k-5487	Eman	22	Malware Analysis	E6	5485	Mr. Asif	0334-5987412
CY3110	21k-4476	Riaz	23	Malware Analysis	E6	5485	Mr. Asif	0334-5987412
CY3105	21k-2559	Moin	20	Blockchain & Cryptography	E1	6582	Mr. Zafar	0333-6987456
CY3105	21k-4879	Zain	23	Blockchain & Cryptography	E1	6582	Mr. Zafar	0333-6987456

The prime attribute is Student_ID as it has unique value in each row.

Functional Dependencies:

FD1: Student_ID → Course_ID, Student_Name, Student_Age, Course_Name, Classroom, Faculty_ID, Faculty_Name, Faculty_Contact

FD2: Course_ID → Course_Name, Classroom (Transitive Dependency so not in 3NF)

FD3: Faculty_ID → Faculty_Name, Faculty_Contact (Transitive Dependency so not in 3NF)

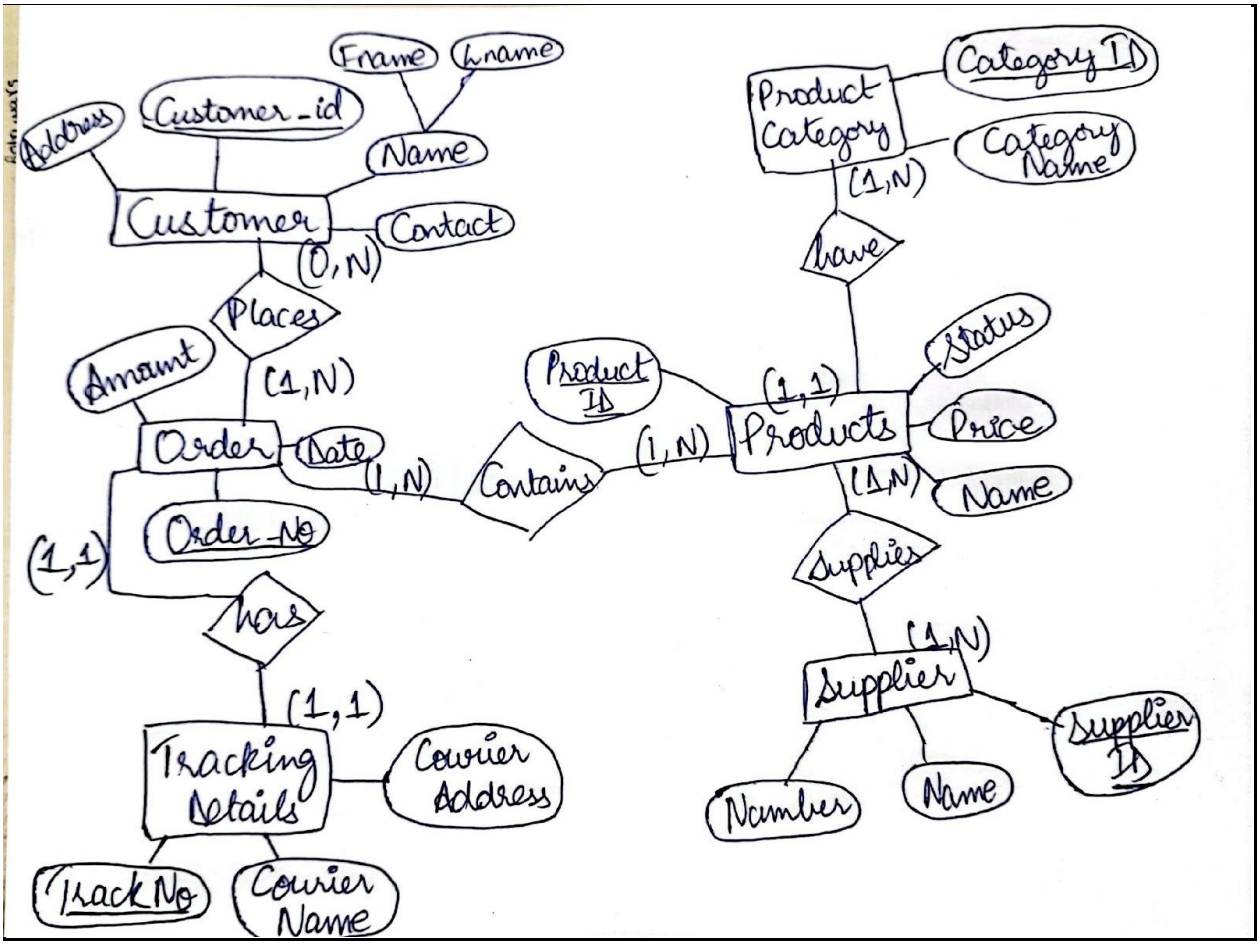
Relations after 3NF:

R1: (Student_ID, Student_Name, Student_Age, Course_ID)

R2: (Course_ID, Course_Name, Classroom)

R3: (Faculty_ID, Faculty_Name, Faculty_Contact, Course_ID)

Answer 5:



Answer 6:

1. πYear, Title(BOOKS)
2. σPublisher = 'McGraw-Hill'∧Year<1990(BOOKS)
3. πStName(σAge>30(STUDENTS)) − πStName(σMajor='CS'(STUDENTS))
4. πStName(σSTUDENTS.StId=borrows.StId (σMajor='CS'(STUDENTS) × borrows))
5. πStName(STUDENTS) − πS1.StName(σS1.Age>S2.Age(ρS1(STUDENTS)×ρS2(STUDENTS)))
6. πTitle(BOOKS) − πB1.Title(σB1.Year>B2.Year(ρB1(BOOKS) × ρB2(BOOKS)))
7. BOOKS ⋈ (πDocId(σKeyword='database'(Descriptions)) ∩ πDocId(σKeyword='programming'(Descriptions)))
   Or

BOOKS ⋈ (Descriptions÷{('database'), ('programming')}) with {('database'), ('programming')}) being a constant relation.

Answer 7:

1. write a SQL query to find all orders generated by London-based salespeople. Return ord_no, purch_amt, ord_date, customer_id, salesman_id.

Ans: SELECT * FROM orders WHERE salesman_id IN (SELECT salesman_id FROM salesman WHERE city='London');

2. write a SQL query to determine the commission of the salespeople in Paris. Return commission.

Ans: SELECT commission FROM salesman WHERE salesman_id IN (SELECT salesman_id FROM customer WHERE city = 'Paris');

3. write a SQL query to find salespeople who had more than one customer. Return salesman_id and name.

Ans: SELECT salesman_id,name FROM salesman a WHERE 1 < (SELECT COUNT(*) FROM customer WHERE salesman_id=a.salesman_id);

4. write a SQL query to find those customers whose grades are different from those living in Paris. Return customer_id, cust_name, city, grade and salesman_id.

Ans: SELECT * FROM customer WHERE grade NOT IN (SELECT grade FROM customer WHERE city='Paris');

5. write a SQL query to calculate the total order amount generated by a salesperson. Salespersons should be from the cities where the customers reside. Return salesperson name, city and total order amount.

Ans: SELECT salesman.name, salesman.city, subquery1.total_amt FROM salesman, (SELECT salesman_id, SUM(orders.purch_amt) AS total_amt FROM orders GROUP BY salesman_id) subquery1 WHERE subquery1.salesman_id = salesman.salesman_id AND salesman.city IN (SELECT DISTINCT city FROM customer);

6. write a SQL query to find those orders where the order amount exists between 500 and 2000. Return ord_no, purch_amt, cust_name, city.

Ans: SELECT a.ord_no,a.purch_amt, b.cust_name,b.city FROM orders a,customer b WHERE a.customer_id=b.customer_id AND a.purch_amt BETWEEN 500 AND 2000;

7. write a SQL query to find the details of an order. Return ord_no, ord_date, purch_amt, Customer Name, grade, Salesman, commission.

Ans: SELECT a.ord_no,a.ord_date,a.purch_amt, b.cust_name AS "Customer Name", b.grade, c.name AS "Salesman", c.commission FROM orders a INNER JOIN customer b ON a.customer_id=b.customer_id INNER JOIN salesman c ON a.salesman_id=c.salesman_id;